CH 3 — System Modelling

Luke Lu • 2026-11-10

We would like to visualize the design of a software system

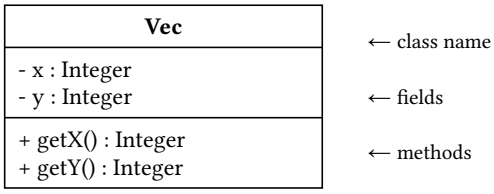
1. Identify key entities/abstraction/classes
2. Relationships between entities

Use Unified Modelling Language

Modeling classes

It can be represented as a box with 3 sections

Example



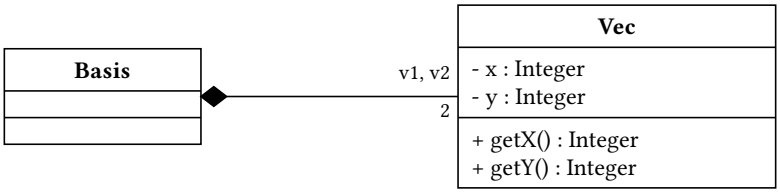
`- ...` means it is `private`

`+ ...` means it is `public`

Relationship 1: Composition/OWNS

```
class Basis{
  Vec v1, v2;
};
```

`Basis` is composed/owns of 2 `Vec` objects



The **filled diamond** ◆ sits on the **owner** (`Basis`); the role names and multiplicity are written at the **owned** end (`Vec`).

A OWNS-A B implies:

- if A is destroyed, B is destroyed
- if A is copied, B is copied

When a class has fields of objects, those objects are owned by the class

Sometimes fields thare pointers to objects might also have a OWNS-A relationship

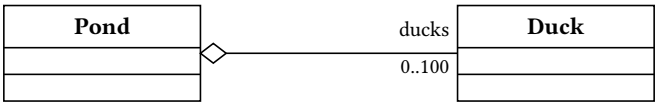
Relationship 2: Aggregation/HAS:

- Aggregation is never represented using fields that are objects.
- Fields that are pointers DO NOT automatically represnet a HAS-A relationship

A HAS-A B implies:

- If A is destroyed, B lives on

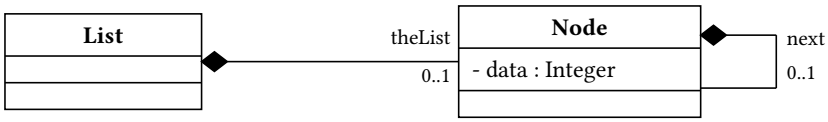
```
class Pond{
  Duck *ducks[100];
}
```



The **hollow diamond** ◇ sits on the container (`Pond`). Multiplicities use `..` (e.g. `0..100`, `0..*`).

Nodes of Lists

```
class List {
  Node *theList;   // List OWNS its nodes
};
class Node {
  int data;
  Node *next;      // each Node OWNS the next one
};
```



Pointer fields here are still OWNS-A: the `List` is responsible for deleting its nodes, and each `Node` for deleting `next`.

Relationshp 3: Specialization/Inheritance/IS

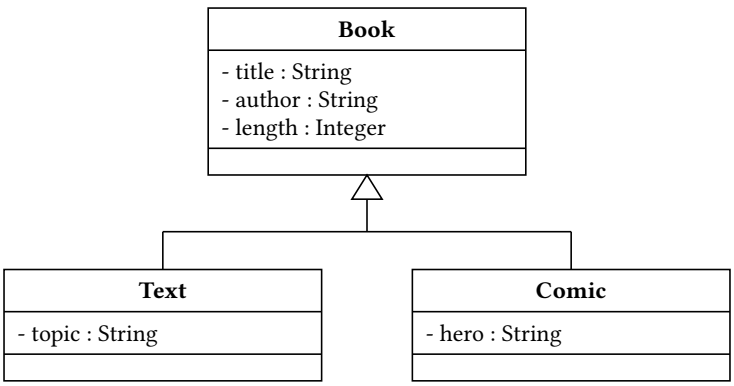
Consider example:

- Book: title, author, length
  - Text: title, author, length, topic
  - COmic: title, author, length, hero

Observe:

Text IS-A Book specialized with an extra topic field,

Comic IS-A Book specialized with an extra hero field



The **hollow triangle** △ points **at the superclass**. Subclasses only list their **new** fields; inherited ones are not repeated.

`Book` is a Parent/Base/Super class

`Comic` and `Text` are Children/Derived/Sub class which inherits all members of `Book` including private, public, and Methods

Any methods that you could call a `Book` object you can call on a `Text` object

Code can be found "lectures/19-inheritance/examples"

```
class Book {
  string title, author;
  int length;
  public:
    Book(string title, string author, int length):
    title{title}, author{author}, length{length}
};

class Text: public Book{
  string topic;

};

class Comic: public Book{
  string hero;
};
```

Here `Text` inherited 3 private fields: `title, author, length`

But `private` means private that is

A `Text` object reserves space for `title, author, length, topic`

Even though `Text` objects have space for these `private` inherited fields, `Text` class cannot access these fields

```
//INVALID
Text::Text(string title, string author, int length, string topic):
    title{title}, author{author}, length{length}, topic{topic};
```

The above INVALID code has 3 errors:

1. Inherited fields are `private`
2. MIL can only initialize fields it declared
3. Steps for object construction of derived classes need superclass constructed first

NEW steps for constructing derived objects:

1. Space is reserved
2. Superclass part of the object is constructed
3. Subclass fields that are objects are constructed/Modeling
4. Constructor body runs

So we need a default constructor of `Book`, but since we wrote our own default constructor for `Book`, we need to call the custom constructor explcitly in `Text`

```
Text::Text(string title, string author, int length, string topic) :
 Book{title, author, length}, topic{topic} {}
```

`protected` members are accessible to inherited subclasses

```
class Book{
  protected:
    string author;
};

void Text::addAuthor(string auth){
  author+= auth;
}

int main(){
  Text t;
  //INVALID, requires public level
  t.author=...;
}
```

Here `Text` can access `authors` b/c it is protected in `Book`

> 💡 **Tip — Advice for choosing Level**
>
> 1. Keep fields private to guarantees invariants
> 2. We could provide `protected` mutators/accessors

Method Overriding

Imagine: I want to answer when different books are heavy.

`Book` is heavy if `lenght>200`

`Text` is heavy if `length>500`

`Comic` is heavy if `length>30`

Code found “lectures/inheritance/example2”

```
class Book{
  int length;
  public:
    int getLength() const {return length}
    bool isHeavy() const {return length > 200;}
};

bool Text:isHeavy() const {return getLength() > 500;}
bool Comic:isHeavy() const {return getLength() > 30;}
```

Method overriding: replacing an inherited method with a new implementation of the method

```
Book b1{"a", "b", 50};
b1.isHeavy(); //Book::isHeavy() runs, gives out `false`
Comic c1{"c", "d", 40, "e"};
c1.isHeavy(); //Comic::isHeavt() runs, gives out `true`

Book b2 = c1; //LEGAL b/c Comic IS-A Book

b2.isHeavy(); //Book::isHeavy() runs, gives out `false`

//`Book` copy constructor ran, treating `c1` as a `Book`
```

Object slicing/coersion:

`Superclass s = subObject;`

The additional fields of `subObject` are dropped when `Superclass`'s copy constructor is called.

We want to avoid this.

```
Comic c1{"c", "d", 40, "e"};
Comic *pc {&c1};

pc->isHeavy(); //Comic::isHeavy() runs, gives out `true`

Book *pb{&c1}; //VALID b/c `Comic` is a `Book`, the reverse is INVALID
//No slicing occurred b/c the pointer is simply pointing to the location of `c1`
pb->isHeavy(); //Book::isHeavy() runs, gives out `false`. Different from Java
```

C++ looks at the declared type of the object/pointer to determine which method to run (by default)

Most often, we want to call the most specialized method based on the actual type of the object involved.

> 📘 **Definition — `virtual`** C++ introduced keyword `virtual`
>
> - Declare the method `virtual` in the base class
> - `override` the method as needed in subclasses, and it forces the compiler to confirm the method is an inherited method, it is not required to be used

Example:

```
class Book{
  ...
  public:
    virtual bool isHeavy() const {...}
};
class Comic: public Book {
  ...
  public:
    bool isHeavy() const override {...}
};

//In main.cc
Comic *pc{&c};
Book *pb{&b};
Book &rb{c};
```

```
pc->isHeavy();
pb->isHeavy();
rb->isHeavy();
//All calls Comic::isHeavy
```

Dynamic Disbatch:

The method is choosen when the program is running based on the runtime type of the object initialized

```
//collection uses polymorphism, a polymorphic array
Book *colleciton[100];
...
collection[0] = new Text()
...
collection[...] = new Comic()
...
for(int i = 0; i<100; ++i){
  cout<< collection[i].isHeavy() <<endl;
}
```

Destructor Revisted

Updated 4 steps to object destruction:

1. subclass destructor runs
2. destructor runs for subclass fields that are objects
3. Superclass part of object is destroyed(superclass destructor called)
4. Space is reclaimed

“lectures/19-inheritance/example5”

```
class X {
  int *x;
 public:
  X(int n): x{new int [n]} {}
  virtual ~X() { delete [] x; } //added virtual
};

class Y: public X {
  int *y;
 public:
  Y(int n, int m): X{n}, y{new int [m]} {}
  ~Y() { delete [] y; }
};

int main () {
  X myX{5}; // Automatically triggering X destructor from Y destructor
  Y myY{5, 10}; //First get reclaimed from Y destructor

  X *p = new Y{5, 10};
  delete p; //Memory leak
  //X::~X() called, the Y part is leaked, solution: make destructor virtual
}
```

> 💡 **Tip — Destructor**
>
> If a class have(or may have) a subclass, declare the destructor `virtual`

`final` Keyword

If a class will never have a subclass, declare it to be `final`

Example: `class Y final: public X{...};`

Pure Virtual Methods

(Goal: Abstract vs Concrete classes)

```
class Student {
  ...
  public:
    virtual int fees()=0; // PURE VIRTUAL METHOD
    //We do not want to implement this, but without =0, it will not compile
};

class regular : public Student{
  public:
    int fees() override {...}
};
class coop : public Student{
  public:
    int fees() override {}
}

//main
Student *allStudent[41000];
allStudent->fees();
```

Virtual: subclass may have different behaviour
Pure Virtual: subclass must provide an implementation to be considered to a concrete class

Abstract vs Concrete

A class is abstract if either

1. It declares a pure virtual method (i.e. Student in the bove example)
2. Inherits a pure virtual method that it does not implement

so that you cannot instantiate object of abstract classes. (i.e. cannot create a Student object in above)

Representation of virtual in UML:

- Virtual/Pure virtual methods: in italics
- Abstract: Name of class //
- Static: underline
- protected: #

C++ Templates

```
template <typename T>
class Stack {
  T *contents;
  int cap;
  int size;
  public:
    Stack();
    ~Stack();
    void push(T x);
    void pop();
    T top();
};

//main

Stack<int> sInts;
sInts.push(1);
Stack<String> sStrings;
sStrings.push("hi");
Stack<Stack<int>> s;
s.push(sInts);
```

C++ supports templates.

A template class is a class parameterized on one or more types.

- To create objects of the template class we need to provide the type as parameter

Standard Template Library (STL)

- Many C++ template classes: `vector, array, set, map, etc.`

`vector` is a dynamic array that automatically resizes as needed.

Example:

```
import <vector>;
vector<int> v{3, 4}; //[3, 4], vector object is on the STACK, automatically destroyed
when going out of scope.
v.emplace_back(5); //[3, 4, 5]
// vector v{3, 4}; is still legal because it is auto inferred, but recommended to
indicate

for(int i = 0; i<v.size();++i){
  cout << v[i] <<endl; //operator[] is overloaded
}

for(vector<int>::iterator it = v.begin(); it!=v.end(); ++it) cout << *it <<endl;
for(auto n: v) cout<<n<<endl;
for(vector<int>::reverse_iterator it = v.rbegin();it != v.rend(); ++it){
  cout<< *it <<endl;
}//reverse loop, rbegin() gives the last item, and rbegin() gives one before first
element

v.pop_back(); //removes last element
```

Many STL class methods take `Iterator` as parameter

Iterator supports some arithmetics:

```
v.erase(v.end()-1);
v.erase(it+3);
```

Remove all 5 from the vector

```
for(auto it = v.begin(); it!= v.end()){
  if (*it ==5) {
    it = v.erase(it); //without it, it will skip the element after the erased element
  }else{
    ++it;
  }
}
```

Note: `erase()` may cause elements to shift internally

- existing iterators may become invalid
- returns an `Iterator` to the next element

Object Oriented Desgin Strategy

- Create abstract classes to specify the interface
- Use pointers of the base class to call interface methods
- To change behaviour, change the ctual objects these pointers points to (virtual methods leading to dynamic dispatch to concrete implementation)

Iterator Desgin Pattern (revisted)

- Create an abstract iterator class

An `AbsIter` can points ot a `List::Iterator` or a `Set::Iterator`

```
class AbsIter{
  public:
    virtual int &operator* const=0;
    virtual AbsIter &operator++() =0;
    virtual bool operator!=(const AbsIter &rhs) const =0;
    virtual ~AbsIter(){}
};

class List{
  ...
  public:
    class Iterator : public AbsIter{
      ...
    };
    auto &start = l.begin();
    foreach(start, l.end(); addFive);
};
class Set{
  ...
  public :
    class Iterator : public AbsIter{
      ...
    };
    auto &start = l.begin();
    foreach(start, l.end(); addFive);
};
```

```
template <typename Fn>;
void foreach(AbsIter &start, const AbsIter &end, Fn f){
  while (start !=end){
    f(*start);
    ++start;
  }
}
```

Observer Pattern

Subjsect/Publisher: generates events/data

Observer/Subscriber: interested in notifications

Subject → Observer Concrete subject ←Concrete observer